TP Broadcasting: Laravel Reverb – Private Channel

Dans ce TP, on va créer un projet de Chat Temps Réel.

1 : Création du projet

```
composer create-project Laravel/Laravel 023-broadcasting-chat-reverb-private
cd 023-broadcasting-chat-reverb-private
php artisan install:broadcasting
composer require laravel/breeze --dev
php artisan breeze:install blade
```

Lancer les serveurs :

```
php artisan serve
php artisan reverb:start
```

2 : Configuration

Dans `.env` :

```
BROADCAST_CONNECTION=reverb
QUEUE_CONNECTION=sync
```

3 : Modèle Message

```
php artisan make:model Message -m
```

Migration :

```
        Schema::create('messages', function (Blueprint $table) {
            $table->id();
            $table->text('content');
            $table->timestamps();
        });
```

```
php artisan migrate
```

4 : Event Broadcasting

```
php artisan make:event PrivateMessageSent
```

Code :

```
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;

class PrivateMessageSent implements ShouldBroadcast
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $message;
    public $userId;
```

```
    public function __construct($message, $userId)
    {
        //
        $this->message = $message;
        $this->userId = $userId;
    }

    public function broadcastOn(): array
    {
        return [
            new PrivateChannel('chat.' . $this->userId),
        ];
    }
}
```

5 : Autorisation des channels

routes/channels.php

```
use Illuminate\Support\Facades\Broadcast;

Broadcast::channel('chat.{id}', function ($user, $id) {
    return (int) $user->id === (int) $id;
});
```

6 : Contrôleur

```
php artisan make:controller ChatController
```

```
    public function index()
    {
        $messages = Message::where('user_id', auth()->id())->get();
        return view('chat', [
            'messages' => $messages,
            'user' => auth()->user()
        ]);
    }

    public function send(Request $request)
    {
        $user = auth()->user();
        $message = Message::create([
            'content' => $request->content,
            'user_id' => $user->id
        ]);

        event(new PrivateMessageSent(
            $request->content,
            $user->id
        ));

        return response()->json(['status' => 'ok']);
    }
```

NB : Ajouter le champ « content » dans la propriété $fillable du modèle Message ;

```
protected $fillable = ['content', 'user_id'];
```

7 : Routes

```
Route::middleware('auth')->group(function () {
    Route::get('/', [ChatController::class, 'index']);
    Route::post('/send', [ChatController::class, 'send']);
});
```

8 : Vue Blade

resources/views/chat.blade.php

```
<!DOCTYPE html>
<html>

<head>
    <title>Chat</title>
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <script src="https://cdn.tailwindcss.com"></script>
    @vite(['resources/js/app.js'])
</head>

<body class="bg-gray-100 p-4">

    <h1 class="text-2xl font-bold text-center mb-4">Chat Temps Réel</h1>

    <div id="messages" class="bg-white p-4 rounded shadow-md mb-4">
        @foreach ($messages as $msg)
            <p class="text-gray-700">{{ $msg->content }}</p>
        @endforeach
    </div>

    <div class="flex items-center space-x-2">
        <input type="text" id="message" placeholder="Votre message"
            class="border border-gray-300 rounded p-2 flex-1">
        <button onclick="sendMessage()"
            class="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600">Envoyer</button>
    </div>

    <script>
        function sendMessage() {

            fetch('/send', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content
                },
                body: JSON.stringify({
                    content: document.getElementById('message').value
                })
            });
            document.getElementById('message').value = '';
        }
      window.userId = {{ auth()->id() }};
```

```
    </script>

</body>

</html>
```

8 : Laravel Echo

resources/js/app.js:

```
window.Echo.private(`chat.${window.userId}`).listen(
    "PrivateMessageSent",
    (e) => {
        let div = document.getElementById("messages");
        div.innerHTML += `<p>${e.message}</p>`;
    }
);
```

9 : Test

Ouvrir **2 navigateurs** :

Créer un utilisateur dans chaque navigateur.
Envoyer un message.
Le message ne doit pas apparaître dans l'autre fenêtre sauf si le même utilisateur.